SyncField: Design and Evaluation of an Offline-First Mobile Data Collection and Synchronization System for Low-Connectivity Environments

Mohamed Tariq Abdel Farraj · Mohammed Sharif · Ghaidaa Al-Sir · Dua al-Mahdi
Information Technology Department, College of Customs Sciences, Medical and Technology
Supervisor: Yahya Othman, MSc
Corresponding author: [email to be inserted before submission]

Abstract

Field data collection is frequently disrupted in environments where Internet connectivity is weak, intermittent, or unavailable. This paper presents SyncField, an offline-first mobile data collection and synchronization system designed to preserve fieldwork locally while maintaining controlled synchronization with a central database. The system was implemented as a Flutter/Dart Android application with SQLite local persistence, secure session storage, a Node.js/Express.js REST backend, and PostgreSQL central storage. The design combines predefined personal and dynamic forms, rule-based validation, visible pending responses, role- and form-based authorization, supervisor scope, user-initiated synchronization, stable client identifiers for idempotency, a durable mutation queue for update, soft-delete, and restore operations, and optimistic versioning with explicit conflict resolution. An engineering design-and-evaluation approach was used, with iterative development guided by Scrum principles and a requirement-traceable test matrix. The final documented evaluation contained 92 functional and integration test cases covering server configuration, authentication and sessions, authorization and scope, offline persistence, synchronization, duplicate prevention, record lifecycle, conflict handling, reporting, localization, security, and privacy. All 92 cases achieved their expected outcomes within the verified Android 13 debug build and trusted local network test environment. The results support the feasibility of the implemented offline-first workflow, while conclusions are limited by Android-focused testing, manual rather than background synchronization, local network backend deployment, and the absence of broad performance, concurrency, disaster-recovery, and long-duration validation.

Keywords—Offline-first; mobile data collection; Flutter; SQLite; synchronization; idempotency; PostgreSQL; access control.

1. Introduction

Organizations that collect structured information in the field often operate where continuous network access cannot be guaranteed. Paper-based collection and network-dependent applications can interrupt field work, delay data availability, require later re-entry, and increase the risk of duplication or transcription error (Elmasri & Navathe, 2016; Coulouris et al., 2012). Offline-first architecture addresses this problem by treating local persistence as the operational path and network transfer as a controlled secondary process rather than a prerequisite for data entry (Kleppmann, 2017; Android Developers, 2024).

However, offline data capture alone is insufficient for an organizational field system. A practical solution must preserve unsynchronized records, validate input, prevent duplicate server-side effects during retries, enforce role and scope boundaries, retain failed operations, support the lifecycle of already synchronized records, and resolve concurrent updates without silently overwriting authoritative data. These concerns motivate SyncField, a mobile system developed to integrate offline collection with controlled governance and central synchronization.

Accordingly, this study addresses the following research question: can an offline-first mobile architecture support reliable structured field data collection under intermittent connectivity while also providing traceable synchronization, organizational authorization, and safe record lifecycle management?

The main contributions of this work are:

- A layered Flutter/SQLite mobile architecture that keeps valid field responses locally available and exposes unsynchronized work through Pending Responses.
- A user-initiated synchronization workflow with stable client identifiers and idempotent processing to reduce duplicate effects on the central database during retries.
- A durable mutation queue for updates to synchronized records, soft deletion, restoration, retry, and optimistic versioning with explicit conflict resolution.
- An authorization model that combines System Administrator, Supervisor, and Data Collector roles with form permissions, record ownership, supervisor assignments, and backend enforcement.
- A requirement-traceable functional and integration evaluation comprising 92 documented test cases in the verified prototype environment.

2. Related Work and Research Gap

Prior work has addressed important parts of offline-capable mobile systems. Viscaino Quito and Serpa Andrade (2022) studied synchronization between offline and online data-collection sessions through a service boundary. Fernandes et al. (2022) proposed OfflineManager for Android, emphasizing offline status, failed-request management, scheduling, and notification. Duggirala (2018) discussed synchronization techniques, consistency, and conflict concerns in offline-first Android applications, while Guda (2025) examined queues, retries, duplication, and ordering in offline-first chat systems. Pothineni (2024) emphasized local persistence, resilience, synchronization, consistency, and conflict handling at the architectural level.

These studies establish the value of local persistence and deferred synchronization, but this study identifies a gap in integrating these concerns with organizational identity, supervisor scope, form-specific permissions, safe record mutations, explicit conflict resolution, attribution, reporting, and bilingual operation in one applied field data workflow. SyncField does not propose a new synchronization algorithm in isolation; its contribution is the integration of offline-first data collection, secure governance, retry safety, record accountability, and centralized oversight.

Study	Focus	Useful contribution	Position relative to SyncField
Viscaino Quito & Serpa Andrade (2022)	Offline/online data transfer	Local/cloud separation and web-service-based synchronization	Different technology stack; does not describe SyncField's integrated identity, scope, lifecycle, and reporting model.
Fernandes et al. (2022)	Offline status management on Android	Failure retention, scheduling, notifications	Uses an Android-specific library and automatic scheduling; SyncField uses Flutter and deliberate manual synchronization.
Duggirala (2018)	Offline-first synchronization techniques	Consistency, deferred synchronization, conflict considerations	Conceptual treatment; SyncField implements stable identifiers and an explicit record lifecycle workflow.
Guda (2025)	Offline-first chat architecture	Queues, retry, duplication, ordering	Targets chat messages rather than structured field records and organizational authorization.
Pothineni (2024)	Offline-first mobile architecture	Resilience, local persistence, synchronization	Broad architecture; SyncField adds roles, form permissions, supervisor scope, reports, attribution, and lifecycle controls.

Table 1. Positioning of SyncField relative to selected related work.

3. Research Method and System Design

3.1 Engineering and Development Approach

The study used an engineering design-and-evaluation approach. The problem and project objectives were translated into functional and non-functional requirements, architecture, implementation components, and requirement-linked tests in accordance with established software-engineering practice (Sommerville, 2016). Development was iterative and incremental using Scrum principles, with successive work on project structure, offline forms and SQLite persistence, internal authentication and secure sessions, roles and permissions, synchronization, record lifecycle and conflict handling, reporting, localization, and quality hardening (Schwaber & Sutherland, 2020; Pressman & Maxim, 2020).

The analyzed system defined 24 functional requirements and 12 non-functional requirements. The non-functional criteria emphasized reliability, data integrity, backend-enforced security and privacy, clear error handling, configuration separation, maintainability, and testability. Quantitative performance thresholds were deliberately excluded when no retained measurements supported them.

3.2 Architecture

SyncField uses a layered architecture. In Flutter, presentation is separated from application logic and data access. SQLite stores operational records and the mutation queue; secure storage holds session data; SharedPreferences stores non-sensitive settings; and ApiClient handles authenticated JSON communication (Flutter Documentation, 2024; SQLite Consortium, 2024). The Node.js/Express.js backend validates authentication, authorization, form permissions, supervisor scope, record ownership, and payloads before executing PostgreSQL transactions through REST-style HTTP/JSON interfaces (Fielding, 2000; Node.js Documentation, 2024; PostgreSQL Global Development Group, 2024).

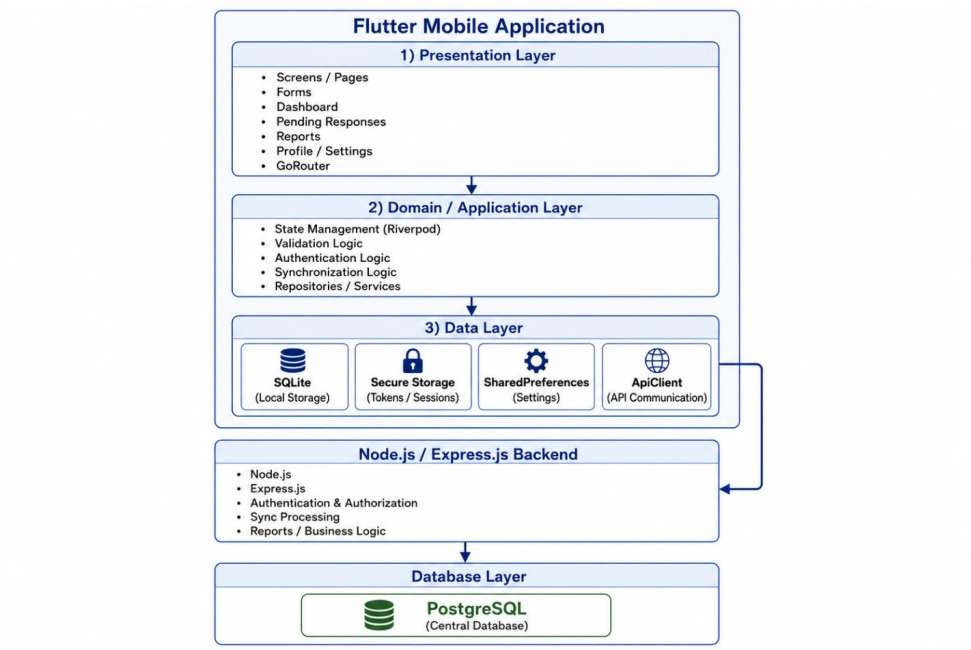

Figure 1. Implemented SyncField architecture (adapted from the graduation research implementation figure).

3.3 Offline-First Data and Synchronization Model

Authorized users can open predefined personal and dynamic forms, validate input, and store valid responses in SQLite without requiring an active connection. A response receives a stable client_response_id and remains locally visible as unsynchronized until an authoritative server result is accepted. Synchronization is explicitly user-initiated through Sync Now; network availability and server reachability are treated as distinct conditions. Failed or uncertain records remain pending for later retry rather than being removed or marked as synchronized.

For already synchronized records, update, soft-delete, and restore actions are persisted as durable local mutations. Each mutation includes a stable client_operation_id and a base server version. The backend applies optimistic version checks: a matching version permits the transaction, while a mismatch produces an HTTP 409 conflict and current server state. The user can then adopt the server version or rebase the local change on the latest version rather than relying on silent Last-Write-Wins behavior.

3.4 Identity, Authorization, and Security Boundaries

The system defines three operational roles: System Administrator, Supervisor, and Data Collector. Access decisions combine role-level permissions, per-form permissions, record ownership, and supervisor assignment scope. Client-side visibility improves usability, but the backend remains the authoritative security boundary; hidden Flutter controls are not treated as sufficient authorization. Sensitive session data are stored through secure storage, while plaintext passwords are not stored in SQLite. The verified deployment uses HTTP only within a trusted local network; public deployment requires HTTPS.

3.5 UML Design Models

To make the system behavior and structure explicit without reproducing every role-specific model from the underlying graduation research, Figures 2–10 present the UML views most directly related to the paper's central contribution. The models summarize actor interaction, application flow, synchronization and conflict behavior, state transitions, software components, core classes, and the evaluated deployment boundary.

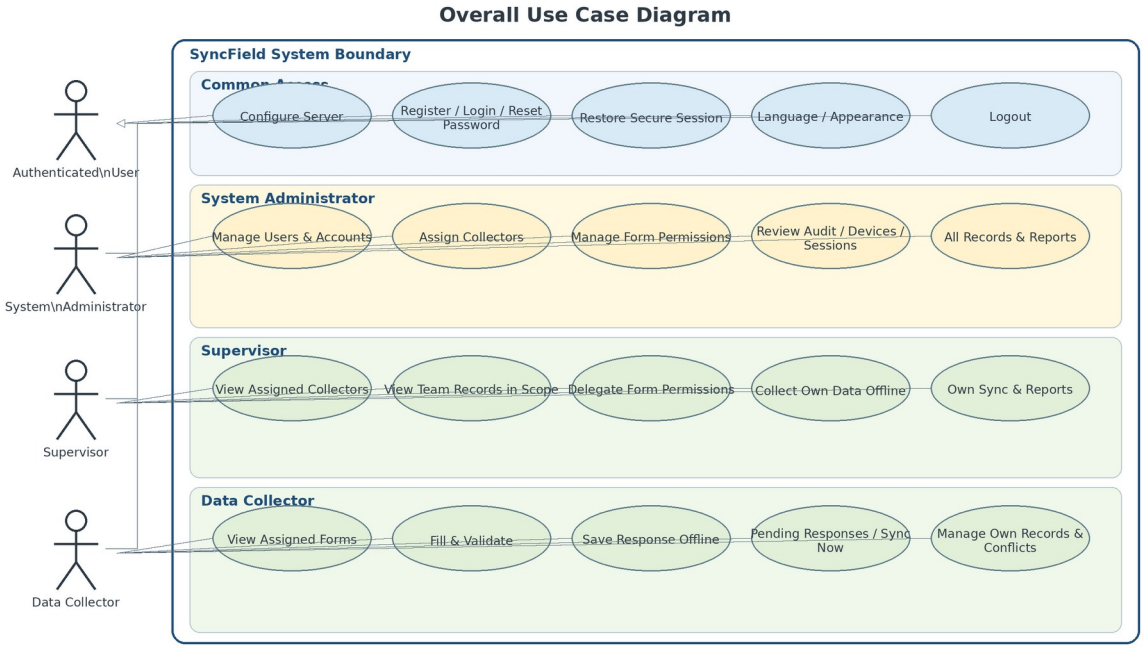

Figure 2. Overall use case diagram for SyncField.

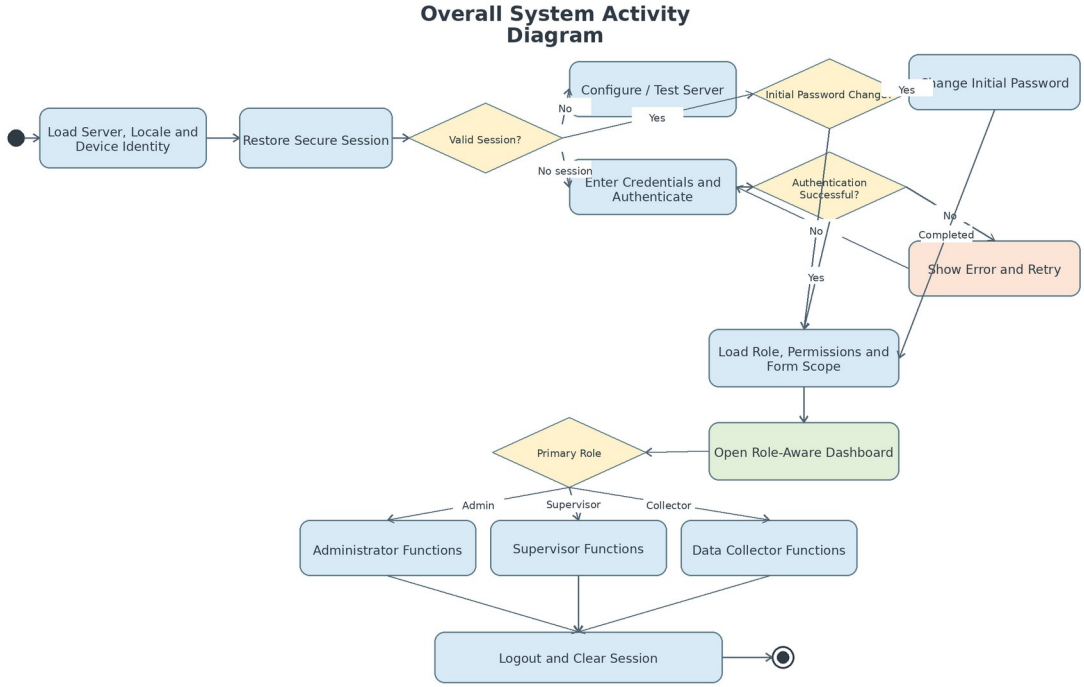

Figure 3. Overall system activity diagram.

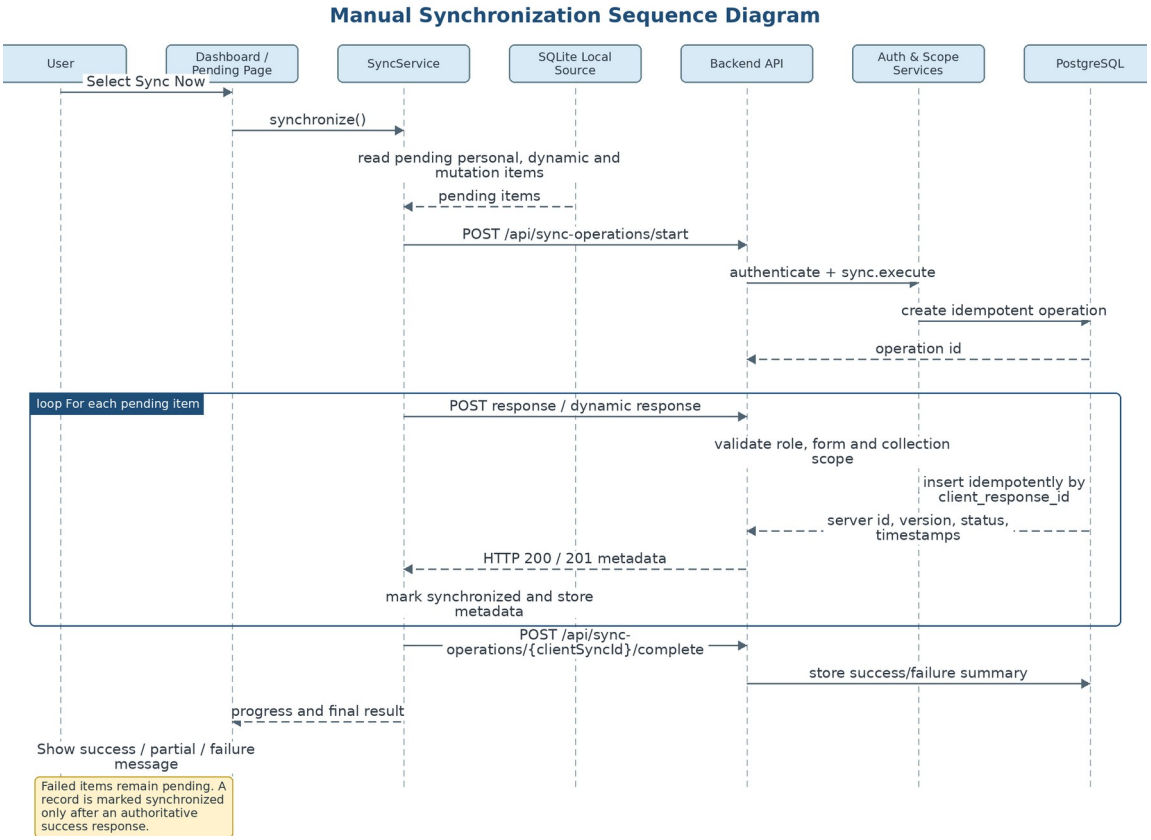

Figure 4. Manual synchronization sequence diagram.

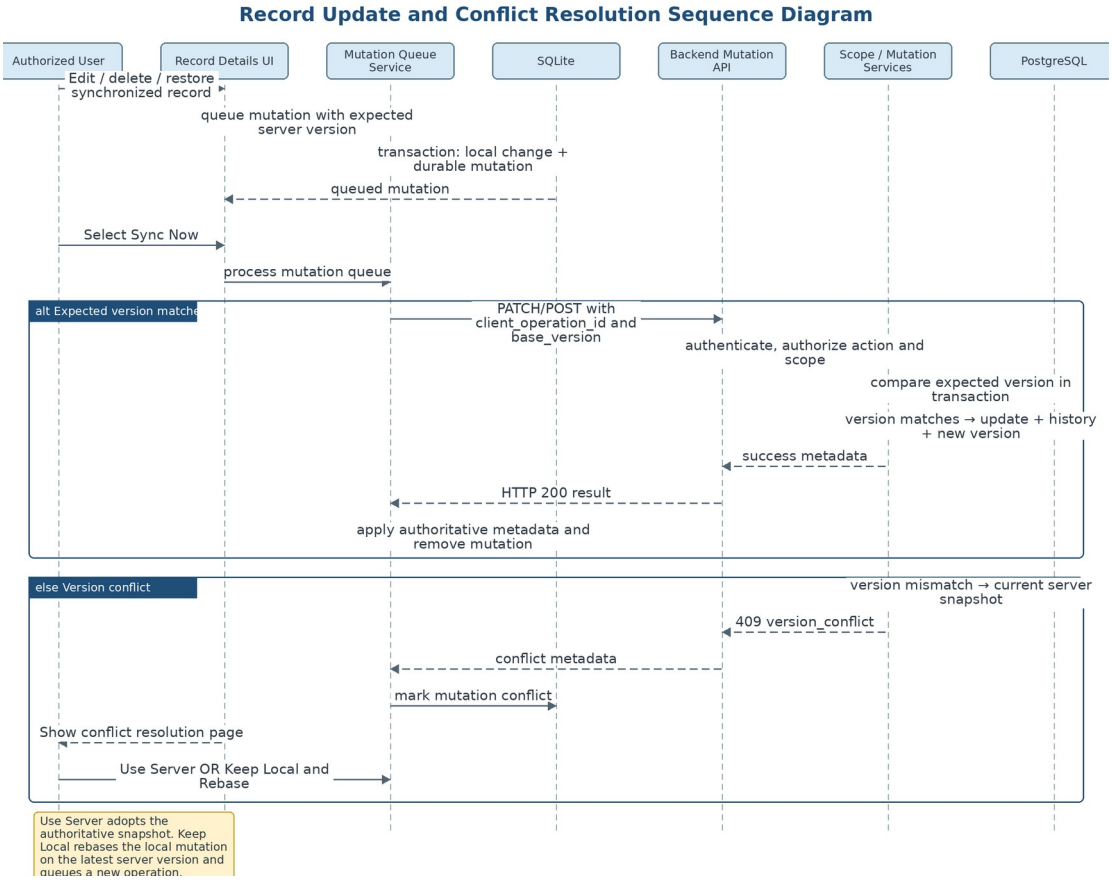

Figure 5. Record update and conflict-resolution sequence diagram.

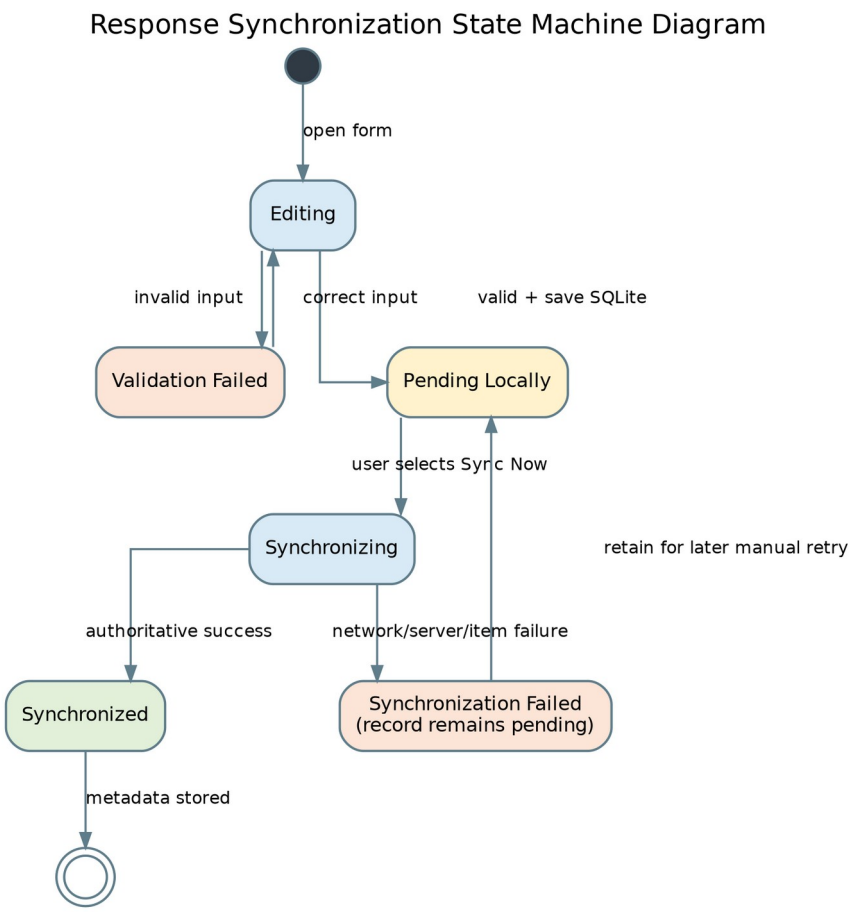

Figure 6. Response synchronization state-machine diagram.

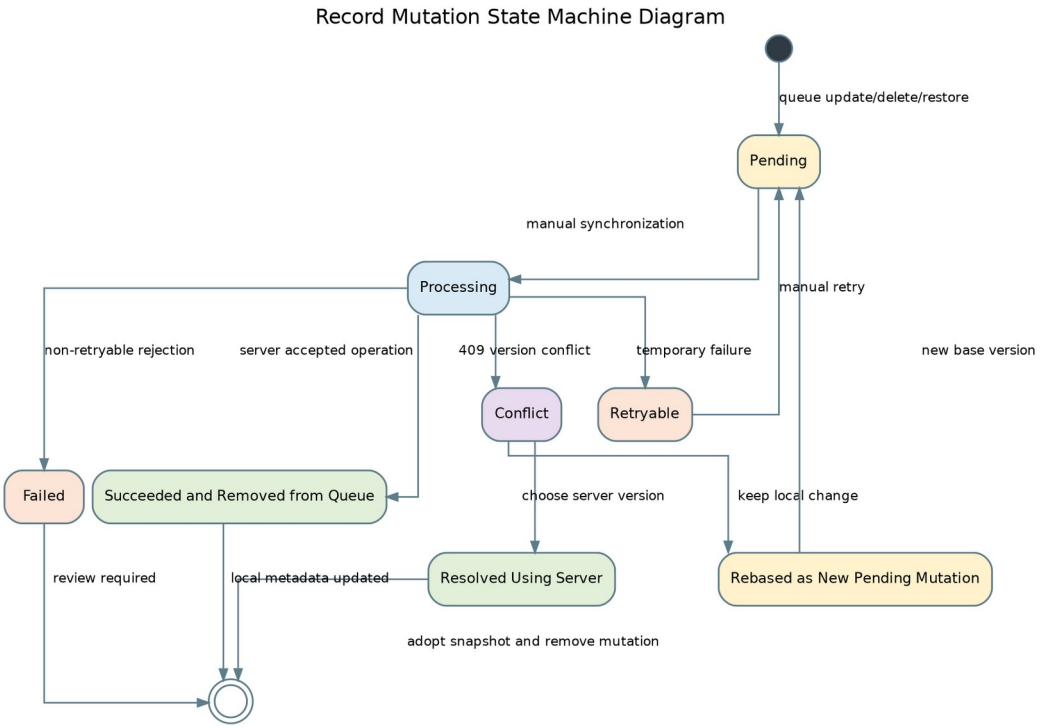

Figure 7. Record mutation state-machine diagram.

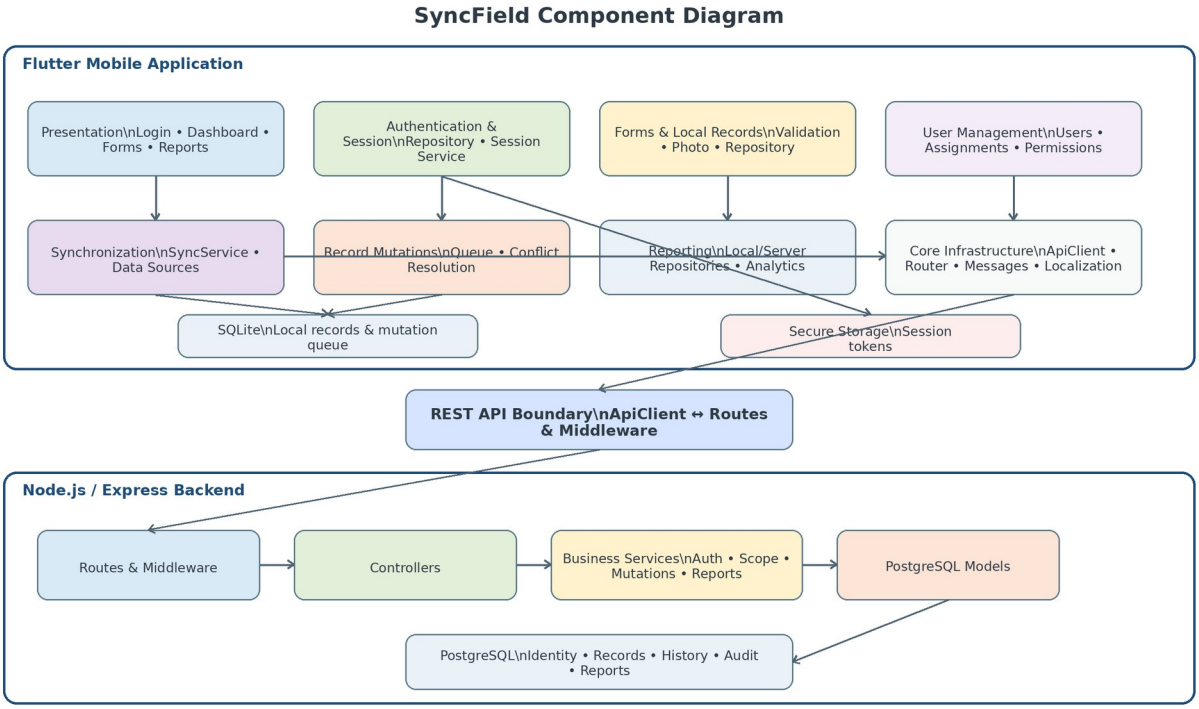

Figure 8. SyncField component diagram.

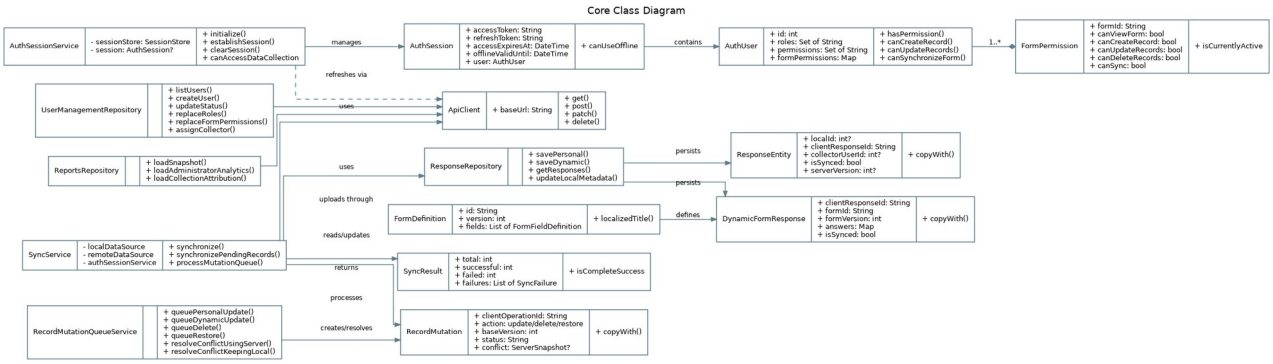

Figure 9. Core class diagram.

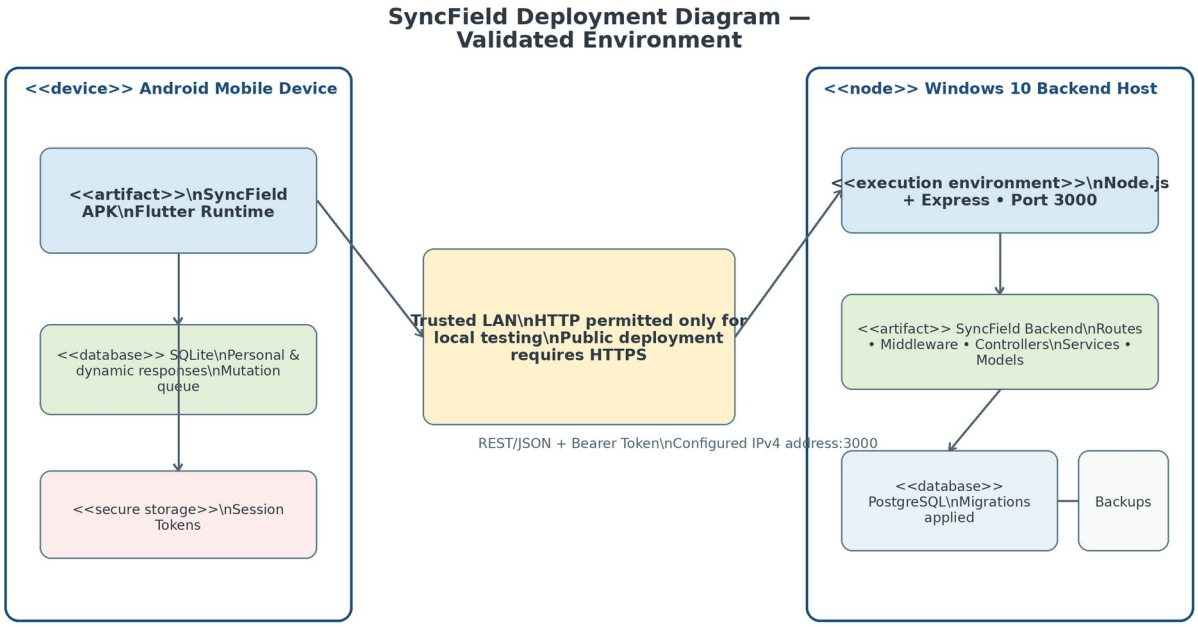

Figure 10. SyncField deployment diagram for the validated test environment.

Together, these diagrams show how offline persistence, explicit synchronization, authorization scope, idempotent operations, optimistic-version conflict handling, and the client-server deployment are connected within one implementable design.

4. Implementation Environment

The evaluated prototype was built as a Flutter application for Android with a locally hosted REST backend. Table 2 summarizes the final environment recorded in the research. The device and software versions are reported to support reproducibility of the documented evaluation rather than to imply support for every Android or server configuration.

Component	Configuration	Evaluation role
Mobile client	Redmi 10C; Android 13; MIUI Global 14.0.9	Real-device functional, interface, offline, and synchronization testing
Application	Debug APK (app-debug.apk), version 1.0.0+1	Evaluated prototype build
Flutter / Dart	Flutter 3.44.8 stable; Dart 3.12.2	Mobile SDK and runtime
Local database	SQLite via sqflite 2.4.2+1; schema version 8	Offline persistence and migration behavior
Secure storage	flutter_secure_storage 10.3.1	Session and token storage
Backend	Node.js 24.13.0; Express.js 5.2.1; port 3000	REST API and business logic
Central database	PostgreSQL 18.4, 64-bit Windows	Authoritative central storage
Host / network	Windows 10 laptop; Wi-Fi/local LAN via Honor X9d hotspot	Connected, disconnected, and server-unreachable conditions

Table 2. Final prototype evaluation environment.

5. Evaluation

5.1 Test Design

Each documented test case was linked to at least one system requirement. Evaluation covered unit and domain behavior, Flutter widgets and routing, integration among mobile repositories, REST services, and PostgreSQL, offline persistence and recovery after restart, authorization and scope enforcement, database and migration behavior, security and privacy controls, and interface inspection. The source research explicitly distinguishes functional verification from unmeasured claims: formal usability conclusions, broad load or concurrency claims, and production-scale performance metrics were not reported without retained raw observations.

Evaluation area	Representative coverage	Recorded outcome
Server configuration	Pre-login settings, validation, reachable/unreachable backend	Expected outcomes achieved

Evaluation area	Representative coverage	Recorded outcome
Authentication and sessions	Registration/login, invalid credentials, initial password, recovery, restoration, refresh, logout, revocation, inactive account	Expected outcomes achieved
Authorization and scope	User management, roles, form permissions, supervisor assignments/delegation, backend rejection of out-of-scope access	Expected outcomes achieved
Offline persistence	Offline form use, validation, local save, restart persistence, Pending Responses, SQLite schema/migration behavior	Expected outcomes achieved
Synchronization	Manual sync, progress, partial/failure retention, timeout, idempotent replay, duplicate prevention	Expected outcomes achieved
Record lifecycle	Queued update/delete/restore, optimistic versions, HTTP 409, Use Server, Keep Local/Rebase, history/audit	Expected outcomes achieved
Reports and localization	Role-scoped reports, attribution, Arabic-English direction, profile/settings, unified messages	Expected outcomes achieved
API/security/privacy	Status handling, authorization, invalid/expired sessions, safe errors/logs, absence of exposed secrets	Expected outcomes achieved

Table 3. Summary of requirement-traceable evaluation areas.

5.2 Results

The final documented matrix contained 92 functional and integration test cases. According to the recorded project outcomes, all 92 achieved their expected results, yielding a functional pass rate of 100% for the executed matrix. This figure should be interpreted as completion of the documented prototype test cases, not as a statistical estimate of reliability under all field conditions.

Offline and local-persistence cases verified that authorized forms remained usable during server unavailability, invalid input was rejected, valid responses were committed locally, pending records survived application restart, and SQLite schema version 8 supported the tested workflow. Synchronization cases verified processing with a reachable backend, retention of work under unreachable and timeout conditions, prevention of overlapping operations, handling of partial outcomes, and safe idempotent replay using stable identifiers. Authentication and authorization cases verified internal login/session behavior, password workflows, account-status handling, role and form permissions, supervisor scope, and backend rejection of deliberate unauthorized or out-of-scope requests. Record-lifecycle tests exercised durable mutations, stale-version detection, explicit conflict handling, and the Use Server and Keep Local/Rebase resolution paths.

Only the functional pass rate is reported quantitatively because the source research retained a documented denominator for the 92-case matrix. Response time, throughput, duplicate rate, offline-persistence rate, authorization-rejection rate, and conflict-resolution rate were intentionally omitted because the required raw sample counts were not retained.

6. Discussion

The evaluation supports the central design premise that an offline-first mobile workflow can preserve structured fieldwork during connectivity loss while keeping synchronization visible and controlled. SQLite local persistence and the Pending Responses mechanism give users an explicit representation of work that has not yet received authoritative server confirmation. This addresses a common ambiguity in network-dependent workflows, where a timeout can leave the user uncertain whether a record was accepted.

The system also demonstrates that offline-first behavior must extend beyond local saving. Stable client identifiers are required for retry safety; backend authorization is required because interface hiding alone is not a security boundary; durable mutation records are needed for changes to already synchronized records; and optimistic version checks are needed to avoid silent overwriting under concurrent updates. These mechanisms align with principles of distributed systems concerning partial failure, transaction integrity, and explicit conflict handling (Coulouris et al., 2012; Bernstein & Newcomer, 2009; Kleppmann, 2017).

Compared with related work that emphasizes individual aspects such as offline status, synchronization, queues, or general resilience, SyncField integrates those concerns with an organizational governance model. The resulting contribution is therefore architectural and applied: a field data workflow in which offline persistence,

synchronization semantics, roles, form permissions, supervisor scope, record history, reporting, and bilingual interfaces are designed as parts of one system rather than independent features.

7. Limitations and Threats to Validity

The reported findings are bounded by the prototype evaluation environment. First, verification focused on one Android device (Redmi 10C, Android 13) and a debug APK; broad device and platform coverage was not established. Second, synchronization was deliberately manual, so the results do not evaluate automatic background synchronization. Third, the backend and PostgreSQL database were hosted on a Windows 10 laptop within a trusted local network; production HTTPS, public-network deployment, centralized monitoring, backup and restore operations, and disaster recovery were not established.

Fourth, the 92/92 result reflects the documented functional and integration matrix and should not be generalized to quantitative performance, scalability, concurrency, or long-duration reliability. The research did not retain sufficient raw measurements for response time, throughput, broad concurrent load, or formal statistical reliability analysis. Fifth, no formal participant-based usability study was conducted. Finally, full SQLite file encryption through SQLCipher, GPS capture, electronic signatures, an administrator form builder, and external social sign-in were outside the implemented scope.

8. Conclusion and Future Work

This paper presented SyncField, an offline-first mobile data collection and synchronization system for low-connectivity environments. The implemented architecture combines Flutter/Dart, SQLite local persistence, secure session storage, a Node.js/Express.js REST backend, and PostgreSQL central storage. It supports rule-based validation, visible pending records, user-initiated synchronization, stable identifiers for retry safety, backend-enforced authorization and supervisor scope, durable mutations for synchronized records, explicit optimistic-version conflict handling, role-scoped reporting, and Arabic-English localization.

The requirement-traceable prototype evaluation reported 92 of 92 expected functional and integration outcomes within the verified Android 13 and trusted local-network environment. These results support the feasibility of the integrated workflow but do not constitute production certification or broad performance validation. Future work should investigate production HTTPS deployment, stronger local-data protection such as SQLCipher, optional background synchronization with controlled retry policies, multi-device and long-duration testing, performance and scalability measurement, backup and disaster-recovery procedures, improved conflict-resolution interfaces, advanced dashboards, and additional field capabilities such as geospatial data and electronic signatures where appropriate.

Acknowledgments

The authors acknowledge Yahya Othman, MSc, for academic supervision and guidance during the development of the graduation project and preparation of this manuscript.

Declarations for Journal Submission

Author contributions: [To be completed before submission according to the target journal's authorship or CRediT policy.]

Conflict of interest: [To be confirmed by all authors before submission.]

Funding: [To be confirmed before submission; no funding statement was provided in the source thesis.]

Ethics and human participants: This study reports software-system testing and did not include a formal participant-based usability study. Confirm the target journal's ethics or institutional-review requirements before submission.

Data and evidence availability: The project evidence package contains the documented testing records. Add a public repository link only if the authors choose to release non-sensitive code or evidence.

References

Android Developers. (2024). Offline-first data layer guidance. https://developer.android.com/topic/architecture/data-layer/offline-first

Bernstein, P. A., & Newcomer, E. (2009). Principles of transaction processing (2nd ed.). Morgan Kaufmann.

Coulouris, G., Dollimore, J., Kindberg, T., & Blair, G. (2012). Distributed systems: Concepts and design (5th ed.). Addison-Wesley.

Duggirala, J. (2018). Data synchronization techniques in offline-first Android applications. International Journal of Science and Research, 7(3), 1968–1971. https://doi.org/10.21275/SR24724162525

Elmasri, R., & Navathe, S. B. (2016). Fundamentals of database systems (7th ed.). Pearson.

Fernandes, S., Lucena, M. G., Pegurin, L. P., Blanco, J. Z., & Lucrédio, D. (2022). OfflineManager: A lightweight approach for managing offline status in mobile applications. In Proceedings of the 16th Brazilian Symposium on Software Components, Architectures, and Reuse (pp. 30–39). https://doi.org/10.1145/3559712.3559717

Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures (Doctoral dissertation, University of California, Irvine).

Flutter Documentation. (2024). Flutter documentation. https://docs.flutter.dev

Guda, V. R. (2025). Implementation of offline-first architectures for Android internet-based chat systems. International Journal of Multidisciplinary Research and Growth Evaluation, 6(3), 1200–1203. https://doi.org/10.54660/.IJMRGE.2025.6.3.1200-1202

Kleppmann, M. (2017). Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems. O'Reilly Media.

Node.js Documentation. (2024). Node.js documentation. https://nodejs.org/docs

Pothineni, S. H. (2024). Offline-first mobile architecture: Enhancing usability and resilience in mobile systems. Journal of Artificial Intelligence General Science, 7(1), 320–326. https://doi.org/10.60087/jaigs.v7i01.387

PostgreSQL Global Development Group. (2024). PostgreSQL documentation. https://www.postgresql.org/docs/

Pressman, R. S., & Maxim, B. R. (2020). Software engineering: A practitioner's approach (9th ed.). McGraw-Hill.

Schwaber, K., & Sutherland, J. (2020). The Scrum guide: The definitive guide to Scrum. Scrum.org.

Sommerville, I. (2016). Software engineering (10th ed.). Pearson.

SQLite Consortium. (2024). SQLite documentation. https://www.sqlite.org/docs.html

Viscaino Quito, A., & Serpa Andrade, L. (2022). Synchronization procedure for data collection in offline-online sessions. In Applied Human Factors and Ergonomics International (Vol. 28, pp. 182–186). AHFE International. https://doi.org/10.54941/ahfe1001461